Real-Time Crane Health Monitoring and Predictive Maintenance System

Description

A predictive maintenance system with a real-time ingestion API. Any client (a sensor gateway, a script or a simulator) can upload a machine's sensor reading to an API route every second, and the system immediately passes it to the machine learning models and returns the result.

Each reading contains a machine ID, timestamp and signals such as hoist load, motor temperature, vibration and duty cycle. The route validates the reading, adds it to that machine's sliding window of recent data, builds features from the window, and scores it with two models. An anomaly detection model flags unusual behaviour as it happens. A remaining-useful-life (RUL) model keeps an up-to-date forecast of time to failure. The response returns the anomaly flag, health score and service forecast straight away, and every reading and result is stored in a database. A fleet dashboard reads from the API and updates live, showing which machines need attention first, active alerts, and a "service needed in X days" forecast for each one.

Models are trained offline on historical degradation data, then used online to score each incoming reading. The project is framed in business terms, not just model accuracy. It reports the trade-off between false alarms and missed faults, the response time per reading, and the estimated downtime avoided compared with a reactive (fix after failure) approach.

Key features

- Upload route that accepts one reading per machine every second
- Batch upload route for sending many readings or a CSV file at once
- Input validation on every reading (types, ranges, required fields)
- Per-machine sliding window with real-time feature engineering (rolling statistics, trends, frequency features)
- Instant anomaly detection and remaining-useful-life prediction on each reading
- Live per-machine health score (0-100) and risk ranking
- Alerts triggered and stored when thresholds are crossed
- Live fleet dashboard with active alerts and service forecasts
- History routes for readings, scores and alerts
- Client script that simulates a fleet of machines uploading every second
- Cost-based evaluation: false-alarm rate vs missed-fault rate, estimated downtime avoided
- Performance metrics: response time per reading and throughput (readings per second)

Data Source

- **Live uploads:** a client script that sends crane signals (hoist load, motor temperature, vibration, duty cycle) for multiple machines every second, with injected faults and gradual degradation
- **Replay of real datasets:** NASA C-MAPSS and/or CWRU bearing data replayed through the same upload route at one reading per second

- **Any other client:** a sensor gateway or device that can send an HTTP request can use the same route

Technologies Used

Language

- Python

Backend and ingestion

- FastAPI (upload routes, REST API, optional WebSocket for live updates)
- Pydantic (reading and response validation)
- SQLAlchemy (ORM)
- PostgreSQL (storage for readings, scores and alerts)

Data processing and feature engineering

- pandas, NumPy (sliding-window features)
- SciPy (signal and frequency features)

Machine learning

- scikit-learn (preprocessing, Isolation Forest / anomaly detection, evaluation)
- XGBoost (remaining-useful-life regression)
- PyTorch LSTM (optional, only if it improves on XGBoost)
- joblib (model persistence and loading for online scoring)

Dashboard

- Streamlit (live-refreshing fleet view, alerts, forecasts)
- Plotly (interactive charts)

Client and simulation

- Python script with httpx / requests (uploads readings every second)

Tooling

- Docker and Docker Compose (API, database, dashboard)
- pytest
- Git / GitHub